Prompt 3 Heuristic Trade-off & Fine-Tuning

Prompt 3: Heuristic Trade-off & Fine-Tuning

You are an expert AI Analog IC Design Assistant. Please read the following instructions carefully and execute the task. Here are your instructions:

Step 1: Mission & Setup

Read the `OLD_Params_and_Specs.txt` (the provided text file) to obtain the current "Safe State" design parameters ($\vec{x}_{safe}$) and their passing specification performance metrics $\vec{y}_{safe}$ to obtain the safe design point ($\vec{x}_{safe}, \vec{y}_{safe}$)

```
================================================================
 FINAL OPTIMIZATION REPORT
================================================================
--- DESIGN PARAMETERS (x_opt) ---
  x[0] = Wd_opt  :=      # Wd_safe     read and store in vector x_safe[0]
  x[1] = Ld_opt  :=       # Ld_safe      read and store in vector x_safe[1]
  x[2]= Wm_opt  :=    # Wm_safe     read and store in vector x_safe[2]
  x[3]=Lm_opt  :=       # Lm_safe       read and store in vector x_safe[3]
  x[4]=Wss_opt  :=    #Wss_safe       read and store in vector x_safe[4]
  x[5]=Lss_opt  :=      #Lss_safe        read and store in vector x_safe[5]
  x[6]=WB_opt:=       #WB_safe       read and store in vector x_safe[6]


--- SPECIFICATION PERFORMANCE ---
  Spec            | Target       | Achieved     | Status
  ------------------------------------------------------------
  Av              |              |              |  [PASS/FAIL]
  UGF             |              |              |  [PASS/FAIL]
  CMRR            |              |              |  [PASS/FAIL]
  PM              |              |              |  [PASS/FAIL]
  Pw              |              |              |  [PASS/FAIL]
  Av_VICM_MIN     |              |              |  [PASS/FAIL]
  Av_VICM_MAX     |              |              |  [PASS/FAIL]
================================================================
```

From the upper part of the table you can read and store $\vec{x}_{safe}$
From the lower part of the table look at the achieved specs column , this is your $\vec{y}_{safe}$
read it and store it as well.

It doesn't matter whether the Target specs part in the `OLD_Params_and_Specs.txt` are achieved or not .
In this file simply ignore the target column in the table completely .
You will only extract the starting design parameters from it to ($\vec{x}_{safe}, \vec{y}_{safe}$) ,
we don't care about achieving the old specs , we only care about achieving the new specs.

Also read the new given specs file `New_Specs.md` .
By looking at the safe state($\vec{x}_{safe}, \vec{y}_{safe}$) and comparing $\vec{y}_{safe}$ to $\vec{y}_{newTarget}$,if you find all the new specs are

already achieved print "all specs already achieved nothing to do " in the chat . If the new specs from the `New_Specs.md` file were not achieved then proceed to the next part .

Your Mission:

If you look closely at the starting safe design point($\vec{x}_{safe}, \vec{y}_{safe}$) extracted from the given text file called : `OLD_Params_and_Specs.txt` and compare $\vec{y}_{safe}$ to $\vec{y}_{newTarget}$ extracted from the `New_Specs.md` file there will be one spec that is not achieved (e.g., increasing UGF or Av , lowering Power Consumption Pw , increasing CMRR). You must achieve this new target *without* causing any of the other passing specs to fail.

Rules:

1. Do not modify or touch any of currently existing files in the project files
2. You are allowed to create your new python files (.py) & Jupyter notebooks (.ipynb) as long as they don't modify anything or affect any of the previously existing files in the project folder .
3. Do not use the Genetic Algorithm for this and don't use any other ready made Optimization Algorithm in Python . You will write a custom Python script using a heuristic step-based search.
4. Limit your total simulation runs to a maximum of **20 evaluations**.
5. Use the `Thesis_optimizer_Sandbox_Copy` python environment.
6. You will also have access to a simulator calling function (Circuit Wrapper) .You will be using it to run simulations .

Including Your Circuit Wrapper to Run Simulations

```
from LTspice_Circuit_Wrappers import evaluate_5t_ota_ltspice
from SpiceFunctions import SpiceEvaluator, SpiceWaveformReader

LTSPICE_PATH = r"C:\Users\Mohamed\AppData\Local\Programs\ADI\LTspice\LTspice.exe"
BASE_DIR = r"C:\Users\Mohamed\Desktop\ML\CAD_projects\Netlists\GF_180nm_tech_work\project"
```

Read the `OLD_Params_and_Specs.txt` file and extract the safe point parametres and store them in python like this :

```
# ============================================================
#  Starting Point extracted from `OLD_Params_and_Specs.txt` file
# ============================================================
####Starting Point X_safe

Ld= # Extract from the `OLD_Params_and_Specs.txt` file
Wd=# Extract from the `OLD_Params_and_Specs.txt` file
Lm=# Extract from the`OLD_Params_and_Specs.txt` file
Wm= # Extract from the `OLD_Params_and_Specs.txt` file
Lss= # Extract from the `OLD_Params_and_Specs.txt` file
Wss= # Extract from the`OLD_Params_and_Specs.txt` file
WB = # Extract from the `OLD_Params_and_Specs.txt` file


####Starting Point Y_safe
```

```
Av_Start=  # Extract from the `OLD_Params_and_Specs.txt` file
UGF_Start=# Extract from the `OLD_Params_and_Specs.txt` file
CMRR_Start=# Extract from the `OLD_Params_and_Specs.txt` file
PM_Start =# Extract from the `OLD_Params_and_Specs.txt` file
Pw_Start=# Extract from the `OLD_Params_and_Specs.txt` file
Av_VICM_MIN_Start=# Extract from the `OLD_Params_and_Specs.txt` file
Av_VICM_MAX_Start=# Extract from the `OLD_Params_and_Specs.txt` file
TGT_VOUT_MIN_Start=# Extract from the `OLD_Params_and_Specs.txt` file
TGT_VOUT_MAX_Start=# Extract from the `OLD_Params_and_Specs.txt` file
```

Read the `New_Specs.md` file and extract the Target Specs and store them in python like this :

```
# ============================================================================
#  Store the target Variables
# ============================================================================
#### SPECS
Av_TG=  # Read from New_Spec.md file
UGF_TG= # Read from New_Spec.md file
CMRR_TG=# Read from New_Spec.md file
Vicmmax_TG=# Read from New_Spec.md file
Vicmmin_TG=# Read from New_Spec.md file
PM_TG=# Read from New_Spec.md file
Pw_TG=# Read from New_Spec.md file
Av_VICM_MIN=Av_TG-2    # (0.8*Linear gain => -2dB in Decibel gain)
Av_VICM_MAX=Av_TG-2   # (0.8*Linear gain => -2dB in Decibel gain)
TGT_VICM_MIN=# Read from New_Spec.md file
TGT_VICM_MAX=# Read from New_Spec.md file
TGT_VOUT_MIN=0.5 # Use this number just to run the Simulations , they are not mentioned
TGT_VOUT_MAX=2.5 # Use this number just to run the Simulations, they are not mentioned


CL= # use the capacitance value that is given in the New_Spec.md file
VDD=  # use the VDD value that is given in the New_Spec.md file
IB= # use the current value that is given in the New_Spec.md file
```

Here is how to use the Circuit wrapper

```
specs=evaluate_5t_ota_ltspice(Wd, Ld, Wm, Lm, Wss, Lss, WB, Ib, CL, Vicm,
                          TGT_VICM_MIN, TGT_VICM_MAX, TGT_VOUT_MIN, TGT_VOUT_MAX,
                          BASE_DIR=BASE_DIR)
```

Step 2: Heuristic Trade-off Rules (The Designer's Map)

The circuit specifications `[Av, UGF, CMRR, PM, Pw, Av_VICM_MIN, Av_VICM_MAX]` are direct functions of the physical parameters `(Wd, Ld, Wm, Lm, Wss, Lss, WB)`
(Ld will not be modified just use the same value from the starting point)

To decide *which* parameter to modify, do not guess or attempt complex calculus. Use the following established heuristic rules. These rules act as a lookup table showing exactly how modifying a specific transistor dimension affects the circuit specifications, including whether that change is beneficial (✅) or detrimental (❌) to our target constraints.

The "INCREASING" Map (Multiplying by 1.05)

- **Increasing Wd:**
    - Increases: UGF (✅), Av (✅), CMRR (✅), Av_VICM_MIN (✅)
    - Decreases: PM (❌)
    - Constant: Pw, Av_VICM_MAX
- **Increasing Wss:**
    - Increases: UGF (✅), Pw (❌)
    - Decreases: Av (❌), CMRR (❌), Av_VICM_MIN (❌)
    - Constant: PM, Av_VICM_MAX
- **Increasing Lss:**
    - Increases: CMRR (✅)
    - Decreases: Av_VICM_MIN (❌)
    - Constant: PM, UGF, Av, Pw, Av_VICM_MAX
- **Increasing WB:**
    - Increases: Av (✅), CMRR (✅), Av_VICM_MAX (✅), Av_VICM_MIN (✅)
    - Decreases: UGF (❌), Pw (✅)
    - Constant: PM
- **Increasing Wm:**
    - Increases: Av_VICM_MAX (✅)
    - Decreases: PM (❌ slightly)
    - Constant: UGF, Av, CMRR, Pw, Av_VICM_MIN
- **Increasing Lm:**
    - Increases: Av (✅)
    - Decreases: PM (❌ significantly), CMRR (❌), Av_VICM_MAX (❌)
    - Constant: UGF, Pw, Av_VICM_MIN

The "DECREASING" Map (Multiplying by 0.95)

- **Decreasing Wd:**

    - Decreases: UGF (❌), Av (❌), CMRR (❌), Av_VICM_MIN (❌)
    - Increases: PM (✅)
    - Constant: Pw, Av_VICM_MAX

- **Decreasing Wss:**
    - Decreases: UGF (❌), Pw (✅)
    - Increases: Av (✅), CMRR (✅), Av_VICM_MIN (✅)
    - Constant: PM, Av_VICM_MAX

- **Decreasing Lss:**
    - Decreases: CMRR (❌)
    - Increases: Av_VICM_MIN (✅)
    - Constant: PM, UGF, Av, Pw, Av_VICM_MAX

- **Decreasing WB:**
    - Decreases: Av (❌), CMRR (❌), Av_VICM_MAX (❌), Av_VICM_MIN (❌)
    - Increases: UGF (✅), Pw (❌)
    - Constant: PM

- **Decreasing Wm:**
    - Decreases: Av_VICM_MAX (❌)
    - Increases: PM (✅ slightly)
    - Constant: UGF, Av, CMRR, Pw, Av_VICM_MIN

- **Decreasing Lm:**
    - Decreases: Av (❌)
    - Increases: PM (✅ significantly), CMRR (✅), Av_VICM_MAX (✅)
    - Constant: UGF, Pw, Av_VICM_MIN

Agent Instruction: Analyze the stretched target.Based strictly on the mapping rules above, logically deduce **one** physical parameter (from $W_d, L_d, W_m, L_m, W_{ss}, L_{ss}, W_B$) that should be modified to improve the target spec without breaking the others.

- Look for a parameter where the target you want to change has a (✅), in other words search the maps for an action where the target you want to change is marked with a (✅).
- Verify that you have enough margin on the specs marked with a (❌) before making the change.
- Determine whether it needs to be increased or decreased (e.g., If a rule says "Increasing WB decreases Pw (✅)", then to lower power, you must increase WB by a small step aka step WB upwards).
- (Contingency Plan) Select a backup parameter and direction in case your primary choice fails to improve the spec during the step-search.

Agent Instruction for Building the Execution Plan:

You must build a multi-step `execution_plan` list containing all parameters that help your target. Order them strictly by safety margin:

1. **Primary Choice (Safest):** Find a parameter that has a (✅) for your target spec, and has NO detrimental effects (❌) on any other specs. If that doesn't exist, pick one where the (❌) affects a specification where we currently have a massive passing margin.
2. **Contingency 1 (Moderate Risk):** A parameter that improves the target but has a (❌) on a specification that is somewhat close to its failure limit.
3. **Contingency 2 (High Risk):** A parameter that improves the target but negatively impacts a very sensitive specification (like PM).

Example of Execution Plan Logic:

If you want to increase UGF, and your current PM is dangerously close to 60°, you should put W_{ss} as your Primary Choice (because increasing W_{ss} doesn't hurt PM), and put W_d as a contingency (because increasing W_d lowers PM).

Step 3: The Adaptive Search Algorithm (Using the Python Tool)

Do not attempt to write your own `while` loops or optimization algorithms. A robust adaptive step-search tool has already been built for you. You must write a Python script that imports this tool and feeds it the correct dictionaries based on your heuristic reasoning.

Tool Import:

```
from Heuristic_Search_Tool import run_adaptive_search
```

Your Python script must strictly follow this structure:

1. **Define** `current_x`: A dictionary of the starting physical parameters ($W_d, L_d, W_m, L_m, W_{ss}, L_{ss}, W_B$) extracted from the `OLD_Params_and_Specs.txt` file.
2. **Define** `static_constants`: A dictionary containing `Ib`, `CL`, `Vicm`, the four TGT constraints, and `BASE_DIR`.
3. **Define** `current_specs`: A dictionary of the starting performance metrics extracted from `OLD_Params_and_Specs.txt`.
4. **Define** `constraints` **(CRITICAL GATEKEEPER RULE):** This dictionary acts as a pass/fail gatekeeper for every step. **You must only put the ORIGINAL safe targets in this dictionary.** Do NOT put your new, stretched target in here, or the tool will instantly fail. Format: `{'Spec_Name': ('operator', limit_value)}`.
5. **Define Your Execution Plan:** Create a list named `execution_plan` containing dictionaries for your Primary Choice and Contingencies based on the Designer's Map.
6. **Execute the Search Loop:** Write a `for` loop to iterate through your `execution_plan`. Call `run_adaptive_search()` in each iteration, passing the wrapper to `simulator_func`. Deduct

`sims_used` from your budget after each run, and `break` the loop if `status == 'Target Reached'` or if the budget reaches zero.

7. **Extract the Results:** Check the `final_result` dictionary. If a safe trial exists, extract and print the winning state. If no safe design was found.
   Example Tool Call Structure:

```
result_1 = run_adaptive_search(
    param_to_change="Wss",        # Your chosen parameter
    direction="decrease",         # "increase" or "decrease"
    target_spec="CMRR",             # The spec you are stretching
    target_value=400.0e-6,        # The new stretched target
    target_operator=">=",         # ">=" for maximize, "<=" for minimize
    current_x=current_x,
    current_specs=current_specs,
    constraints=constraints,      # MUST CONTAIN ALREADY ACHIEVED TARGETS ONLY
    static_constants=static_constants,
    budget=20                    #  Maximum number of simulation runs you are allowed to do
)
```

Another Example To learn from :

```
# --- EXAMPLE SCRIPT FOR THE AGENT ---

# 1. Define the starting physical parameters (x_safe)

current_x = {
    'Wd': Wd,    # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Ld': Ld,  # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Wm': Wm,# Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Lm': Lm,  # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Wss': Wss,  # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Lss': Lss,  # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'WB': WB   # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
}




# 2. Define the static constants required by the LTspice wrapper
static_constants = {
    'Ib': IB,           # use the current value that is given in the New_Spec.md file from step
1
    'CL': CL,     # use the capacitance value that is given in the New_Spec.md file from step 1
    'Vicm': Vicm,
    'TGT_VICM_MIN': TGT_VICM_MIN,  # Read from New_Spec.md file from step 1
```

```
    'TGT_VICM_MAX': TGT_VICM_MAX,# Read from New_Spec.md file from step 1
    'TGT_VOUT_MIN': TGT_VOUT_MIN, # Use the number used to just to run the Simulations from
step 1
    'TGT_VOUT_MAX': TGT_VOUT_MAX, # Use the number used to just to run the Simulations from
step 1
    'BASE_DIR': r"C:\Users\Mohamed\Desktop\ML\CAD_projects\Netlists\GF_180nm_tech_work\project"
}

# 3. Define the current passing specs (from the OLD file)
current_specs = {
    'UGF': UGF_Start, # Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'PM': PM_Start,# Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Av':Av_Start,# Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'CMRR':CMRR_Start,# Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Pw': Pw_Start,# Extracted from the `OLD_Params_and_Specs.txt` file in step 1
    'Av_VICM_MIN': Av_VICM_MIN_Start,# Extracted from the `OLD_Params_and_Specs.txt` file in
step 1
    'Av_VICM_MAX': Av_VICM_MAX_Start# Extracted from the `OLD_Params_and_Specs.txt` file in
step 1
}




# 4. Define the Hard Constraints (The Gatekeeper)
# Define only the ALREADY ACHIEVED SPECS IN HERE
# NOTE: The target spec (CMRR) is NOT included here!
constraints = {
    'PM': ('>=',PM_TG),
    'UGF': ('>=', UGF_TG),
    'Av': ('>=', Av_TG),
    'Pw': ('<=', Pw_TG),
    'Av_VICM_MIN': ('>=', Av_VICM_MIN_TG),
    'Av_VICM_MAX': ('>=', Av_VICM_MAX_TG)
}
# Note that since CMRR is the unachieved target in this example its target is not used in the
constrains dictionary ,just remove the unachieved target from your constrains .


# 5. Define Your Execution Plan (Primary + Multiple Contingencies)
# Create a list of parameters to try based on the Designer's Map
execution_plan = [
    {"param": "Lss", "direction": "increase"}, # Primary choice
    {"param": "Wd",  "direction": "increase"}, # Contingency 1
    {"param": "WB",  "direction": "increase"}  # Contingency 2
]
```

```python
# 6. Execute the Search Loop
budget_left = 20
final_result = None

for attempt in execution_plan:
    if budget_left <= 0:
        print("Simulation budget exhausted!")
        break

    print(f"\nExecuting Adaptive Search to Maximize CMRR to 120 dB using {attempt['param']}...")
    result = run_adaptive_search(
        simulator_func=evaluate_5t_ota_ltspice,  # <--- Passing the wrapper as a variable here
        param_to_change=attempt['param'],  # Your chosen parameter
        direction=attempt['direction'],  # "increase" or "decrease" that parameter
        target_spec="CMRR", # The spec you are stretching
        target_value=120,    # The new stretched target
        target_operator=">=",   #>= Maximize             #<= Minimize
        current_x=current_x,
        current_specs=current_specs,
        constraints=constraints,
        static_constants=static_constants,
        budget=budget_left
    )

    budget_left -= result['sims_used']
    final_result = result # Save the latest result to extract later

    if result['status'] == 'Target Reached':
        print(f"*** Goal achieved with {attempt['param']}! ***")
        break  # Stop searching, we hit the target!
    else:
        print(f"\n{attempt['param']} failed or plateaued. Sims left: {budget_left}. Moving to next contingency...")




# 7. Extract and Print the Winning Results
if final_result and final_result['safe_trials_log']:
    best_trial = final_result['safe_trials_log'][-1] # Get the last logged successful trial

    print("\n" + "=" * 65)
    print(" ⏳ BEST PARAMETERS FOUND ⏳")
```

```
    print("=" * 65)
    for param, val in best_trial['params'].items():
        if param in ['Wd', 'Wm', 'Wss', 'WB', 'Ld', 'Lm', 'Lss']:
            print(f"  {param} = {val:.4e}")

    print("\n" + "=" * 65)
    print(" 🎯 FINAL SPECIFICATIONS 🎯")
    print("=" * 65)
    for spec, val in best_trial['specs'].items():
        print(f"  {spec}: {val}")
    print("=" * 65)
else:
    print("\n" + "=" * 65)
    print(" ❌ NO SAFE DESIGN FOUND ❌")
    print("=" * 65)
    print("All parameters failed the 1% improvement threshold or violated constraints.")
    print("The specified target is likely outside the physical limits of this topology.")
```

Step 4: Execution & Reporting

Execute your Python script. The `run_adaptive_search` tool will automatically handle the step-size logic, evaluate the constraints, and stop when the target is achieved.

Once it finishes running, check the terminal output:

- **If a winner is found:** Extract the final parameters and specs.
- **If NO SAFE DESIGN is found:** State that the target is physically unreachable with this topology and report the baseline safe specs.
- . Report the results in the chat and write them to a new file named `Final_Fine_Tuned_Report.txt`.

Your output format must strictly match this:

```
======================================================================
 TRADE-OFF FINE TUNING REPORT
======================================================================
--- HEURISTIC REASONING ---
Target: [State the stretched goal]
Chosen Parameter(s): [State the parameter(s) used in your execution plan]
Reasoning: [1 sentence explaining WHY based on the Designer's Map]
--- BEST FINE-TUNED PARAMETERS (x_best) ---
  x[0] = Wd  :=
  x[1] = Ld  :=
  x[2] = Wm  :=
  x[3] = Lm  :=
  x[4] = Wss :=
  x[5] = Lss :=
  x[6] = WB  :=
```

```
--- Specification Performance ---
  Spec            | Target       | Achieved     | Status
  ------------------------------------------------------------
  [Target Spec]   | [New Target] |              |  [PASS/FAIL]
  Av              |              |              |  [PASS/FAIL]
  UGF             |              |              |  [PASS/FAIL]
  CMRR            |              |              |  [PASS/FAIL]
  PM              |              |              |  [PASS/FAIL]
  Pw              |              |              |  [PASS/FAIL]
  Av_VICM_MIN     |              |              |  [PASS/FAIL]
  Av_VICM_MAX     |              |              |  [PASS/FAIL]
================================================================
```